Lab 9: Pipelined Processor

Digital Design and Computer Architecture: RISC-V Edition (Harris & Harris, Elsevier © 2021)

Objective

In this lab, you will complete a pipelined RISC-V processor.

1. Pipelined RISC-V Processor

a) Write a hierarchical SystemVerilog description for the pipelined RISC-V processor and call the module `riscv`. The processor should be compatible with the top-level module below. It should support these instructions: `lw, sw, add, sub, and, or, slt, addi, andi, ori, slti, beq`, and `jal`. Remember that you can use the building blocks from the single-cycle processor HDL in Lab 6 (the single-cycle processor). Modify the testbench from Lab 6 to test your processor using the test program (riscvtest.s and riscvtest.txt) from Lab 6.

b) Design the Hazard Unit shown in Figure 1 for the pipelined RISC-V processor. Use a hierarchical SystemVerilog description to implement your design.

Figure 1 shows the complete pipelined processor. Here is the top-level module declaration:

```
module top(input  logic        clk, reset,
           output logic [31:0] WriteDataM, DataAdrM,
           output logic        MemWriteM);

  logic [31:0] PCF, InstrF, ReadDataM;

  // instantiate processor and memories
  riscv riscv(clk, reset, PCF, InstrF, MemWriteM, DataAdrM,
                      WriteDataM, ReadDataM);
  imem imem(PCF, InstrF);
  dmem dmem(clk, MemWriteM, DataAdrM, WriteDataM, ReadDataM);
endmodule
```

2. Extend Your RISC-V Processor

Describe how to extend the pipelined RISC-V processor to handle the load upper immediate instruction, `lui`. Name any new control signals. Mark up copies of Figure 1 for changes to the datapath and Tables 1 and 2 for changes to the ALU Decoder and Main decoder. Describe any other changes that are required.

Extend the SystemVerilog description for the pipelined RISC-V processor from part 1 to handle the `lui` instruction. Modify the testbench and test program (riscvtest.s and riscvtest.txt) to test your enhanced processor.

Please refer to previous labs for debugging hints.

What to Turn In

1. Please indicate how many hours you spent on this lab. This will be helpful for calibrating the workload for next time the course is taught.
2. A marked-up version of Figure 1 showing the needed modifications for `lui`.
3. Amended Table 1 and Table 2 for changes made to support `lui`.
4. Modified test program and testbench for `lui.`
5. Simulation waveforms (in the order listed above: `clk, reset, PCF, InstrF, MemWriteM, DataAdrM, WriteDataM`, and `ReadDataM`) – all displayed in hexadecimal for ease of reading). Circle or highlight the waves showing that the correct value is written to the correct address, and make sure it is legible.
6. Demo of passing testbench to TAs during lab sections. No late demos.
7. RTL Viewer schematics.
8. Hierarchical SystemVerilog for your top-level processor module (and submodules) matching the declaration given above. Submit your code to a dedicated submission portal (Code of Lab 9) on GradeScope.

Please indicate any bugs you found in this lab manual, or any suggestions you would have to improve the lab.

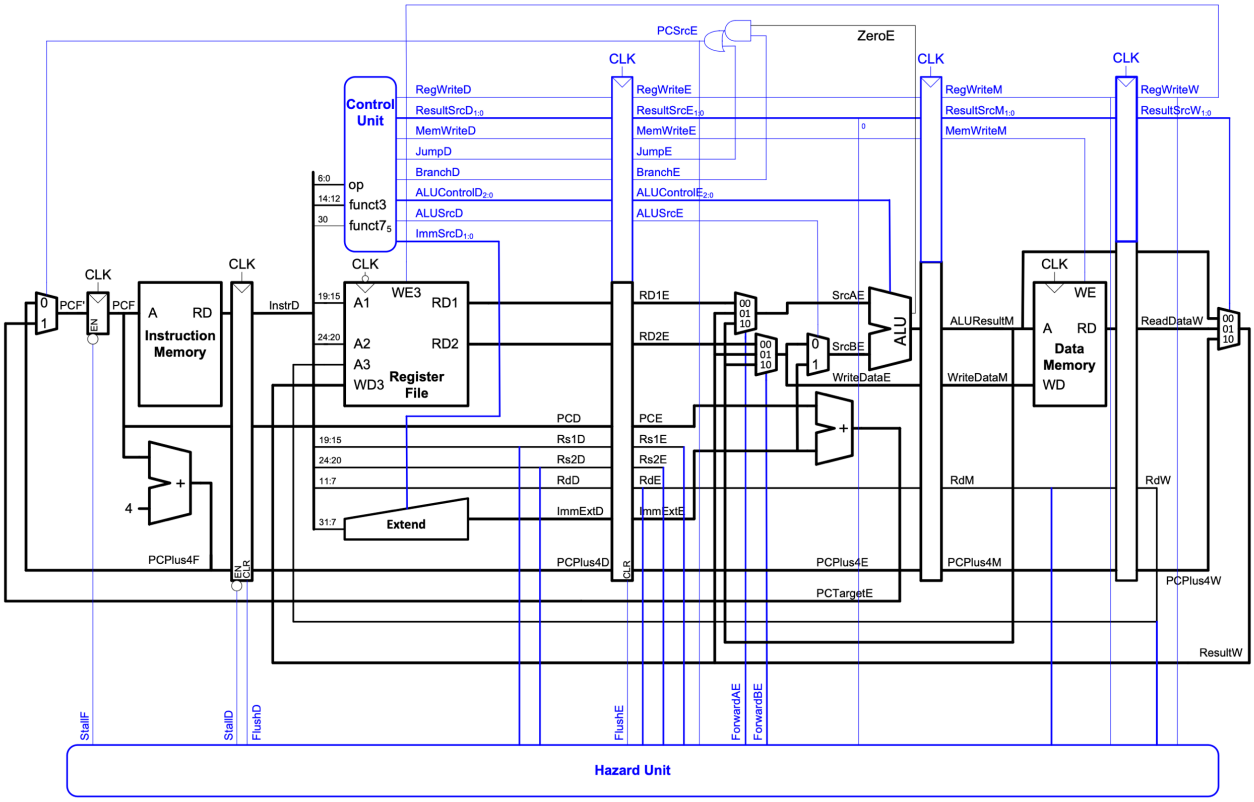

Figure 1: Pipelined Processor with a Hazard Unit.

Table 1. Main Decoder Truth Table

Instruction	Opcode	RegWrite	ImmSrc	ALUSrcA	ALUSrcB	MemWrite	ResultSrc	Branch	ALUOp	Jump
lw	0000011	1	00	0	1	0	01	0	00	0
sw	0100011	0	01	0	1	1	xx	0	00	0
R-type	0110011	1	xx	0	0	0	00	0	10	0
beq	1100011	0	10	0	0	0	xx	1	01	0
I-type ALU	0010011	1	00	0	1	0	00	0	10	0
jal	1101111	1	11	x	x	0	10	0	xx	1

Table 2. ALU Decoder Truth Table

$ALUOp_{1:0}$	$funct3_{2:0}$	$\{op_5, funct7_5\}$	$ALUControl_{2:0}$	Operation
00	x	x	000	Add
01	x	x	001	Subtract
10	000	00, 01, 10	000	Add
	000	11	001	Subtract
	010	x	101	SLT
	110	x	011	OR
	111	x	010	AND